

**BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE**

SPECIALIZATION Computer Science

DIPLOMA THESIS

Pseudo++: A Comprehensive Pseudocode Toolchain for Romanian Secondary Education

Supervisor
Assoc. Prof. Rareș Florin Boian, PhD

Author
Bujor Mihai Alexandru

2026

Abstract

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Motivation	1
1.3 Related Work	1
1.4 Contributions	1
1.5 Thesis Structure	1
2 Related Work	2
2.1 General-Purpose Online Code Editors	2
2.2 Pseudocode-Specific Tools	2
2.3 Educational Language Interpreters	2
2.4 Transpilation in Educational Contexts	2
2.5 Comparison with the Present Work	2
3 Theoretical Background	3
3.1 Formal Languages and Grammars	3
3.2 Syntactic Analysis and Incremental Parsing	3
3.3 Interpretation, Compilation, and Transpilation	3
3.4 Equivalence of Repetitive Control Structures	3
3.5 WebAssembly	3
3.6 Program State Snapshots for Reversible Debugging	3
4 Design and Implementation	4
4.1 System Architecture	5
4.2 Language Grammar	5
4.3 Source Normalization	5
4.4 Parsing and AST Construction	5
4.5 Interpreter	5
4.5.1 Value Representation	5

4.5.2	Execution Environment	5
4.5.3	Expression Evaluation	5
4.5.4	Control Flow	5
4.6	Debugger	5
4.7	Transpiler	5
4.7.1	Code Generation for C, C++, and Pascal	5
4.7.2	Loop Equivalence Transformations	5
4.8	I/O Abstraction and WebAssembly Integration	5
4.9	Web Editor	5
4.10	VSCode Extension	5
5	Evaluation	6
5.1	Testing Methodology	6
5.2	Unit Testing	6
5.3	Integration Testing on BAC Examination Subjects (2020–2025)	6
5.3.1	Dataset Description	6
5.3.2	Results	6
5.4	Transpiler Correctness Verification	6
5.5	Performance Analysis	6
5.6	Discussion	6
6	Conclusions	7
6.1	Summary	7
6.2	Limitations	7
6.3	Future Work	7

Chapter 1

Introduction

1.1 Problem Statement

1.2 Motivation

1.3 Related Work

1.4 Contributions

1.5 Thesis Structure

Chapter 2

Related Work

2.1 General-Purpose Online Code Editors

2.2 Pseudocode-Specific Tools

2.3 Educational Language Interpreters

2.4 Transpilation in Educational Contexts

2.5 Comparison with the Present Work

Chapter 3

Theoretical Background

3.1 Formal Languages and Grammars

3.2 Syntactic Analysis and Incremental Parsing

3.3 Interpretation, Compilation, and Transpilation

3.4 Equivalence of Repetitive Control Structures

3.5 WebAssembly

3.6 Program State Snapshots for Reversible Debugging

Chapter 4

Design and Implementation

4.1 System Architecture

4.2 Language Grammar

4.3 Source Normalization

4.4 Parsing and AST Construction

4.5 Interpreter

4.5.1 Value Representation

4.5.2 Execution Environment

4.5.3 Expression Evaluation

4.5.4 Control Flow

4.6 Debugger

4.7 Transpiler

4.7.1 Code Generation for C, C++, and Pascal

4.7.2 Loop Equivalence Transformations

4.8 I/O Abstraction and WebAssembly Integration

4.9 Web Editor

4.10 VSCode Extension

Chapter 5

Evaluation

5.1 Testing Methodology

5.2 Unit Testing

5.3 Integration Testing on BAC Examination Subjects (2020–2025)

5.3.1 Dataset Description

5.3.2 Results

5.4 Transpiler Correctness Verification

5.5 Performance Analysis

5.6 Discussion

Chapter 6

Conclusions

6.1 Summary

6.2 Limitations

6.3 Future Work